Portfolio Risk Analyst Chatbot: Quantitative Metrics, Retrieval-Augmented Guidance, and Conversational Orchestration

DSA4265 — Group 5

April 18, 2026

Name	Matric Number	Email
Liu Jingyan	A0258209Y	e0969181@u.nus.edu
Low Shi-ya Amelia	A0259379H	e0970351@u.nus.edu
Lee Hui En	A0258545U	e0969517@u.nus.edu
Chang Xiao Xuan	A0256919M	e0967891@u.nus.edu
Fun Wen Xin	A0259409U	e0970381@u.nus.edu
Tan Jun Kiat	A0253051U	e0958639@u.nus.edu

Abstract

Risk indicators shown in consumer applications are frequently opaque with respect to the portfolio weights to which they pertain, whilst unconstrained dialogue with large language models can yield numerically specific yet unsubstantiated statements drawn from parametric memory. This report describes a Streamlit-based conversational system that mitigates these issues by coupling user-supplied holdings to a reproducible analytics pipeline. Validated ticker-weight pairs drive a single download of split- and dividend-adjusted closing prices via Yahoo Finance (`yfinance`); consistent simple-return series underpin annualised volatility, a historical one-day loss quantile used as a VaR-style descriptor, maximum drawdown, a Sharpe-style ratio with the risk-free rate fixed at zero, mean pairwise correlation, and a Herfindahl concentration measure. User utterances are classified with Google Gemini into task types (holistic review, named statistics, definitional queries, and related categories); a deterministic orchestration layer then invokes quantitative computation, hybrid retrieval over local knowledge corpora (BM25 plus dense vectors in Chroma), or both, and formats the user-visible reply as Markdown without a separate generative rewriting stage in the reference interface. The architecture modularises analytics and retrieval to admit further risk measures or forecasting components; findings are contingent on vendor data quality and reporting delay, and the software is offered for research and pedagogical use only, not as regulated investment advice.

Contents

1 Introduction **3**
1.1 Motivation and problem statement . . . 3
1.2 Objectives and scope . . . 3
1.3 Contributions . . . 3
1.4 Report structure . . . 4

2 Data and preprocessing **4**
2.1 Notebook and runtime data pathways . . . 4
2.2 Instrument universe and calendar . . . 4
2.3 Price field and motivation for adjustment . . . 5
2.4 Exploratory visualisation . . . 5
2.5 Cleaning rule and export . . . 5
2.6 Reproducibility . . . 7

3 Methodology **7**
3.1 Portfolio construction and notation . . . 7
3.2 Risk and performance metrics (current build) . . . 7
3.3 Machine-learning components and limitations . . . 8
3.4 Retrieval-augmented generation . . . 8
3.5 Agent and intent orchestration . . . 8

4 System design and implementation **9**
4.1 Architecture overview . . . 9
4.2 User interface . . . 10
4.3 Key modules . . . 11
4.3.1 Data Tools . . . 12
4.3.1.1 Module composition . . . 12
4.3.1.2 `valid_tickers` . . . 12
4.3.1.3 `valid_weights` . . . 12
4.3.1.4 `fetch_price_data` . . . 13
4.3.1.5 `calculate_returns` . . . 13
4.3.1.6 System-level rationale . . . 13
4.3.2 Main Workflow Tools . . . 13
4.3.2.1 `classify_intent` . . . 14
4.3.2.2 `route_and_execute` . . . 16
4.3.2.3 `generate_explanation` (called inside `route_and_execute`) 18
4.3.3 Quant Tools . . . 21
4.3.3.1 Inputs . . . 21
4.3.3.2 Covariance Matrix . . . 21
4.3.3.3 Risk Metrics . . . 21
4.3.3.4 Benchmark Labelling . . . 22
4.3.3.5 Outputs . . . 22
4.3.3.6 Caching . . . 22
4.3.4 LSTM Tools . . . 22
4.3.5 RAG Tools . . . 24

5 Evaluation **25**
5.1 LSTM Evaluation . . . 25
5.2 RAG Retrieval Evaluation . . . 27
5.3 LLM Evaluation . . . 27
5.3.1 `intent_classifier` . . . 28
5.3.1.1 Evaluation Methodology . . . 28
5.3.1.2 Results . . . 29
5.3.2 `generate_explanation` . . . 29
5.3.2.1 Evaluation Methodology . . . 29
5.3.2.2 Results . . . 30

6 Discussion **31**
6.1 Strengths . . . 31
6.1.1 Data Preprocessing . . . 31
6.1.2 Architecture . . . 31
6.1.3 Quantitative Risk Module . . . 32
6.1.4 LSTM Forecasting Module . . . 32
6.1.5 RAG Retrieval Module . . . 32
6.1.6 User Interface . . . 32
6.2 Weaknesses . . . 32
6.2.1 Data Preprocessing . . . 32
6.2.2 RAG Retrieval Model . . . 32
6.2.3 User Interface . . . 33
6.2.4 Explanation Layer . . . 33
6.3 Limitations . . . 33
6.3.1 Data Preprocessing . . . 33
6.3.2 Architecture . . . 33
6.3.3 LSTM Forecasting Module . . . 33
6.3.4 RAG Retrieval Model . . . 34
6.3.5 Explanation Layer . . . 34
6.4 Ethics and responsible use . . . 34

7 Conclusion **34**
7.1 Key Outcomes . . . 34
7.2 Possible Future Improvements . . . 34
7.2.1 Architecture . . . 34
7.2.2 LSTM Forecasting Module . . . 35
7.2.3 RAG Retrieval Model . . . 35
7.2.4 Explanation Layer . . . 35
7.2.5 User Interface . . . 35
7.3 Closing Remarks . . . 35

1 Introduction

1.1 Motivation and problem statement

Portfolio risk is inherently multidimensional: volatility, tail behaviour, co-movement among holdings, and weight concentration capture distinct facets of exposure, and user information needs vary accordingly. At the same time, interactive assistants based on large language models (LLMs) do not, by default, observe an end-user's asset allocation; numerical summaries produced without an explicit data channel therefore cannot be guaranteed to correspond to any stated portfolio.

A second concern is *provenance*. Even high-quality natural-language explanations of financial concepts may be difficult to audit when they are generated solely from model weights rather than from retrievable evidence aligned to the question and, where relevant, to the user's positions.

1.2 Objectives and scope

The objective of the present work is a conversational interface through which, subject to prior validation of the portfolio specification, a user may obtain (i) an aggregated risk summary, (ii) values for implemented scalar risk statistics, (iii) definitional or contextual material on finance terminology, and (iv) short follow-up clarification anchored in the preceding system response. "What-if" and rebalancing-oriented questions are treated within the same analytical family as broad risk-review requests; the implementation records whether the allocation has changed relative to earlier session state.

The scope is instructional and prototypical. Live order execution, personalised advice governed by securities regulation, tax treatment, and production-grade risk reporting are excluded. Prices are taken as end-of-day vendor quotes adequate for coursework and proof-of-concept use, not as tick-level or certified reference data.

1.3 Contributions

The work is primarily engineering-centred; its contributions are architectural and empirical rather than theoretical:

- A Streamlit application with structured portfolio capture, validation logic for tickers and weights, and session-persistent chat transcriptions via Streamlit session state.
- A unified data-access layer for adjusted closing prices, optionally augmented with a broad equity market proxy (SPY) on a common trading calendar, together with centralised return construction interfaces (simple and logarithmic) to standardise preprocessing contracts across analytical and prospective machine-learning components.
- A baseline quantitative module operating on historical daily returns, yielding annualised volatility, a historical one-day quantile usable as a rudimentary value-at-risk figure, a zero risk-free Sharpe-style ratio, maximum drawdown on the cumulative return path, average pairwise correlation among constituents, and a Herfindahl index of weights; these inputs map to a rule-based composite score and categorical risk band for narrative presentation.

- Structured intent inference (Google Gemini) coupled to explicit routing code that invokes the metric pipeline, attaches optional retrieval results, or surfaces recent assistant output for follow-up turns.
- A retrieval-augmented generation (RAG) subsystem comprising multiple curated corpora (including conceptual and strategy-oriented material), intent-conditioned source selection, hybrid lexical and dense-vector retrieval in Chroma, and instrumentation suitable for offline evaluation metrics such as recall at fixed cut-offs.

Systematic human-factor evaluation and full ablation studies are deferred to section 5; the exposition below foregrounds design rationale and implementation fidelity.

1.4 Report structure

section 2 specifies data provenance and preprocessing. section 3 formalises notation, summarises the implemented risk statistics, and outlines RAG and conversational control flow. Subsequent sections treat system realisation (section 4), evaluation protocol and empirical results (section 5), limitations and responsible use (section 6), and concluding remarks (section 7).

2 Data and preprocessing

This section aligns with `data_sourcing_and_preprocessing.ipynb`: group tickers, `Adj Close`, start date, the bounded forward-fill cleaning rule, and exports under `data/processed/`. In parallel, the deployed app downloads prices via `yfinance` using the shared helper in the `agent_tools` package over a rolling five-year window at runtime (simple returns on the analytics path). The notebook outputs remain the canonical basis for figures and for any workflow that reads the CSVs directly.

2.1 Notebook and runtime data pathways

To avoid conflating the fixed report dataset with the live application path, Table 1 distinguishes the two data flows used in this project. Unless otherwise stated, the remainder of this section refers to the notebook pipeline, because it produces the checked-in processed files and the report figures.

2.2 Instrument universe and calendar

The group's default panel comprises **AAPL, MSFT, TSLA, SPY, TLT, GLD**, giving a mix of single-name equities, a broad U.S. equity ETF, long Treasuries, and gold, so correlation structure is non-trivial. Prices are taken from Yahoo Finance via `yfinance`. The preprocessing notebook pins `START = "2020-01-01"` and `END = None` (through "today" at run time). The checked-in `dataset_meta.json` reports 1567 aligned price rows and 1566 return rows after cleaning; re-running the notebook will update those counts if the calendar advances. In deployed inference, `SPY` may also be auto-appended when absent to support benchmark-dependent metrics.

Comparison dimension	Offline notebook preprocessing pipeline	Online deployed application pipeline
Primary purpose	Reproducible data preparation for report figures and downstream offline analysis	Live analytics for user queries in the Streamlit interface
Date window	Fixed by `START = "2020-01-01"` and `END = None`	Rolling five-year lookback computed at runtime
Instrument set	Default group panel: `AAPL`, `MSFT`, `TSLA`, `SPY`, `TLT`, `GLD`	User-supplied tickers, with `SPY` optionally appended for benchmark-aware metrics
Outputs	Persisted CSV and metadata artifacts under `data/processed/`	In-memory price and return matrices for the current session

Table 1: Comparison of the notebook preprocessing pipeline and the deployed application data pipeline.

2.3 Price field and motivation for adjustment

All downstream risk statistics operate on **adjusted** closing levels so that splits and cash dividends do not introduce spurious jumps. Raw `Close` would bias volatility, drawdown, and correlation estimates. The notebook normalises vendor output to a single date index $\times$ tickers table before exploratory analysis.

2.4 Exploratory visualisation

Before committing to a cleaning rule, the notebook inspects level and return series: drawdown from the running high (stress on crisis periods), rolling annualised volatility (short and long windows), and sample path plots of daily simple returns. Figures 1 and 2 reproduce two diagnostics from that lineage on the **current** processed CSVs (`adj_close_prices.csv`, `simple_returns.csv`): drawdown curves per ticker and the correlation matrix of cleaned simple returns (the heatmap labelled for export in the notebook).

2.5 Cleaning rule and export

The following extract matches the preprocessing notebook: forward-fill a *bounded* number of consecutive missing observations, then drop any row that still contains a gap across tickers. Bounded imputation limits silent invention of prices; dropping incomplete rows enforces a common calendar for portfolio sums.

```python
def clean_prices(prices: pd.DataFrame, ffill_limit: int = 5) → pd.DataFrame:
    px = prices.sort_index()
    px = px.ffill(limit=ffill_limit)
    px = px.dropna(how="any")
    return px

prices_clean = clean_prices(raw_prices)
simple_rets_clean = prices_clean.pct_change().dropna()
log_rets_clean = np.log(prices_clean).diff().dropna()
```

Source: `data_sourcing_and_preprocessing.ipynb`.

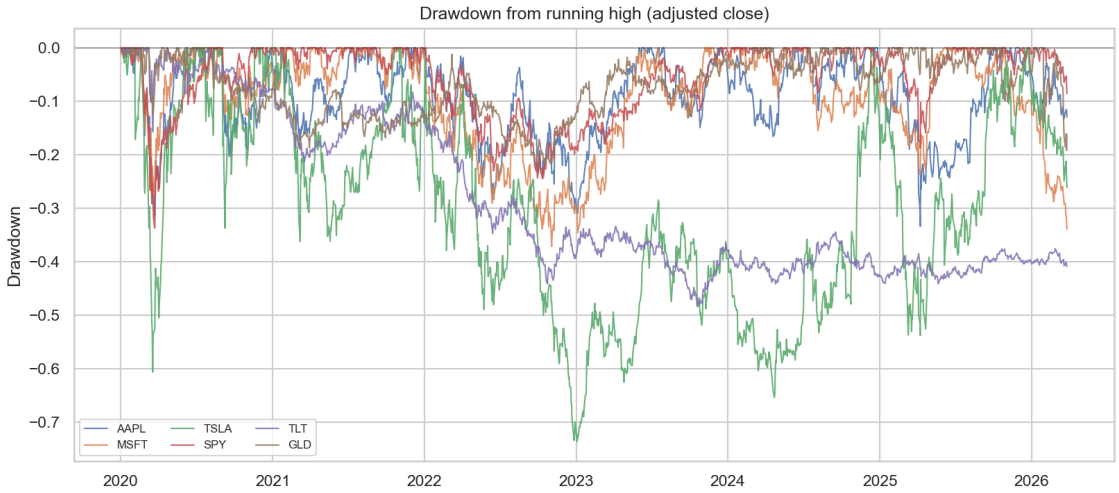

Figure 1: Drawdown from running high on adjusted closes (processed panel). Same construction as the exploratory block in `data_sourcing_and_preprocessing.ipynb`.

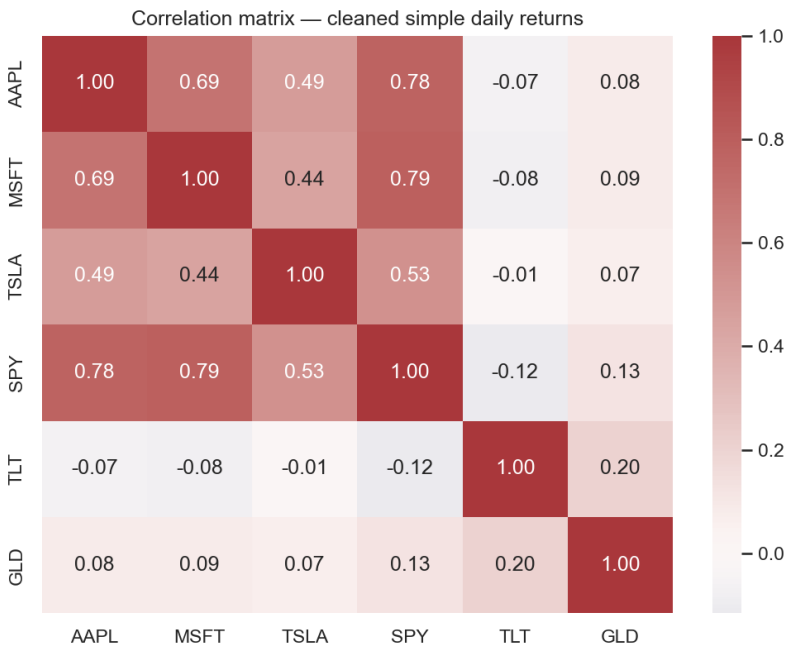

Figure 2: Sample correlation matrix of cleaned simple daily returns. Regenerated from `data/processed/simple_returns.csv` for inclusion in this report.

The notebook writes `adj_close_prices.csv`, `simple_returns.csv`, `log_returns.csv`, and `dataset_meta.json` under `data/processed/`. Other subsystems (offline quant, RAG tabular hooks, LSTM notebooks) can depend on those paths without re-downloading. These processed artifacts also serve as the common input layer for the return notation and risk metrics formalised in Section 3.

2.6 Reproducibility

To support reproducibility, the data preparation workflow should be run from the repository root so that file paths resolve consistently. Because Yahoo Finance data may change over time, the execution date should be recorded whenever figures are regenerated. Figures 1 and 2 can be rebuilt from the processed CSVs by running `python` on `report/scripts/make_data_figures.py`.

3 Methodology

3.1 Portfolio construction and notation

Consider n risky assets. Let $r_t \in \mathbb{R}^n$ denote the vector of simple returns on trading day t, and let $w \in \mathbb{R}^n_{\geq 0}$ denote portfolio weights satisfying $\sum_i w_i = 1$ after normalisation. The portfolio return is

$$r_{p,t} = w^\top r_t.$$

Vectors w are supplied exclusively from the validated interface state and are not inferred by the language model, thereby avoiding implicit reweighting such as undocumented equal-weight assumptions.

3.2 Risk and performance metrics (current build)

The implemented pipeline computes a broader diagnostic set, but six core quantities feed the portfolio-level risk scoring layer:

- **Annualised volatility.** The sample standard deviation of $\{r_{p,t}\}$, multiplied by $\sqrt{252}$ under the conventional mapping from daily to annual units for equities.
- **Historical quantile loss.** The empirical 5th percentile of $\{r_{p,t}\}$ at a one-day horizon, reported without scaling to multi-day regulatory value-at-risk horizons; it constitutes a descriptive tail measure rather than a full loss-distribution model.
- **Sharpe ratio.** The ratio of annualised portfolio return in excess of a fixed 4% risk-free rate to annualised volatility. This is a simplified reference value rather than a user-specific financing-rate adjustment.
- **Maximum drawdown.** Computed on the compounded wealth process $\prod_s (1+r_{p,s})$ as the largest relative decline from a running maximum to a subsequent trough.
- **Average pairwise correlation.** The arithmetic mean of off-diagonal entries in the sample correlation matrix of asset simple returns.
- **Concentration (Herfindahl).** $\sum_i w_i^2$, equal to unity under perfect concentration and to $1/n$ under equal weighting.

These six scalars enter a weighted rule-based mapping to a composite score in $[0, 1]$ and to ordered risk categories (`Low`, `Medium`, `High`). Thresholds are chosen for demonstration and narrative coherence; they are not calibrated against regulatory templates.

In addition, the quant layer computes supporting diagnostics for explanation and follow-up queries, including conditional value at risk, volatility of volatility, Sortino ratio, beta against `SPY`, skewness, excess kurtosis, and per-asset risk contribution. These enrich the explanatory layer, but they are distinct from the six inputs used in the risk-scoring rule set.

3.3 Machine-learning components and limitations

The current system combines deterministic portfolio analytics with a pre-trained LSTM module for forward-looking signals. In the deployed workflow, the LSTM is invoked for full-analysis and trend-prediction intents to produce a forecast volatility level, a directional prediction (`Up`/`Down`), and an associated confidence score. By contrast, the portfolio risk bands (`Low`/`Medium`/`High`) are not produced by a trained classifier; they remain rule-based outputs derived from the six-metric composite score described above.

This separation is intentional. Forecasting is treated as an auxiliary predictive module, whereas the current risk label remains directly auditable from deterministic calculations and explicit thresholds. Any future expansion toward learned risk taxonomies would require a fuller treatment of training data design, temporal leakage control, validation protocol, and calibration.

3.4 Retrieval-augmented generation

The RAG subsystem partitions background knowledge into thematic stores (including, inter alia, ticker disambiguation aids, firm-level summaries, macroeconomic indicators, curated definitions, and strategic-allocation notes). Retrieval is conditioned on the inferred dialogue intent: only subsets of corpora are queried for a given turn. Retrieval itself employs a hybrid of lexical matching (Okapi BM25) and dense retrieval of sentence-transformer embeddings persisted in Chroma. Retrieved chunks then pass through relevance filtering and citation attachment before being exposed to the explanation layer. Logging supports retrospective measurement of retrieval quality (e.g. recall at fixed k and mean reciprocal rank).

In the **reference Streamlit deployment** (`chatbot.py`), deterministic Python assembles the structured context: validated holdings, computed metrics, benchmark labels, risk contributions, optional forecast outputs, recent chat history, and retrieved knowledge-base excerpts. A Gemini explanation model is then called once to generate the final user-facing response from that grounded context. Relative to unconstrained generation, this design narrows-without eliminating-hallucination risk by constraining the model to locally supplied evidence and explicitly authorised numerical values, subject to corpus coverage and chunk boundaries.

3.5 Agent and intent orchestration

Each non-empty user utterance receives a primary intent label from Gemini among holistic portfolio analysis, single-metric interrogation, conceptual explanation, trend or outlook

questioning, conversational follow-up, and general dialogue. Optional secondary intents and normalised entity fields (metrics, concepts) may accompany the classification.

Dispatch logic implemented in Python branches on the primary label. Computationally intensive paths reuse a single price fetch and return construction step where feasible; explanatory paths may issue retrieval calls with intent tags recognised by the RAG layer (`full_analysis`, `concept_explanation`, `trend_prediction`). Follow-up handling relies on recent session history rather than long-horizon memory. Separation of probabilistic intent classification from deterministic numerical execution is intentional: portfolio arithmetic remains auditable code, while the language model is used primarily for routing and explanation rather than for financial calculation itself.

4 System design and implementation

4.1 Architecture overview

The portfolio risk analyst combines agent-style intent handling with fixed execution workflows. It first interprets the user's query, then routes the request into a predefined sequence of function calls rather than allowing free-form tool selection by the LLM.

This design prioritises reliability over autonomy. In a system with clear computation dependencies, skipped or reordered steps can silently corrupt downstream analysis. Fixed workflows ensure that shared computations run in the correct order, improve debuggability and testability, and guarantee that the explanation layer receives a complete and consistent context.

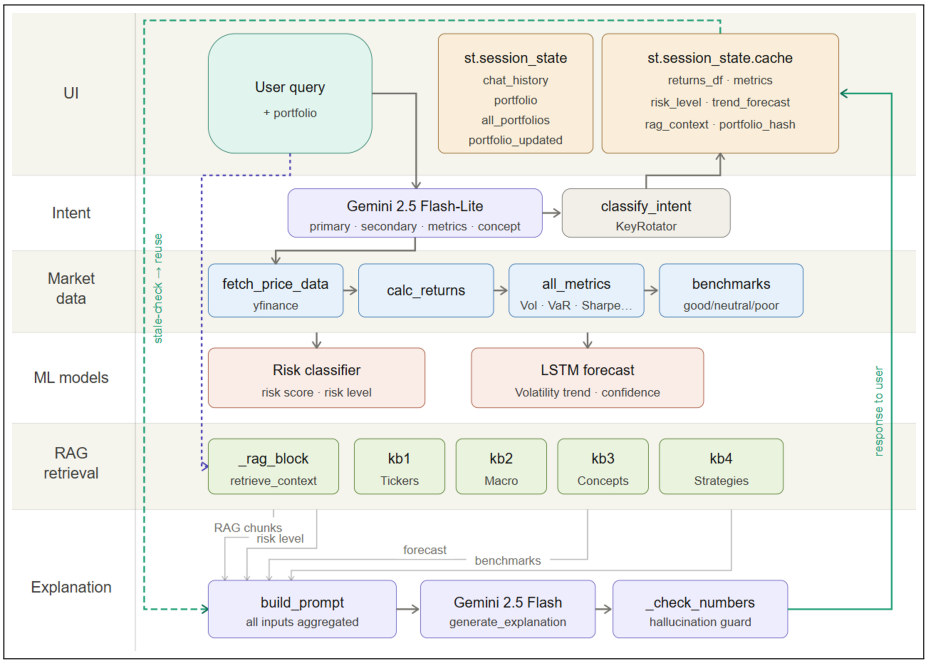

Figure 3: Diagram illustration of the overall chatbot architecture.

As shown in Figure 3, the system is organised into several layers. At the **UI layer**, the user submits a query and portfolio through Streamlit. The **intent layer** classifies the message into one of six intents and extracts any referenced metrics or concepts.

The request is then routed through a **shared computation pipeline** managed by `route_and_execute`. This pipeline computes whatever each intent requires and reuses cached results whenever the portfolio is unchanged.

Furthermore, the pipeline comprises four main components:

- A **data layer** that validates input tickers and extracts the price and return data required by the other components.
- A **quant component** that produces 12 scalar metrics - including portfolio volatility, VaR, Sharpe ratio, beta, and skewness - plus per-asset risk contribution from a covariance matrix computed once and reused downstream.
- An **ML component** combining a rule-based risk classifier and an LSTM forecasting model.
- A **RAG layer** that takes the user query and portfolio tickers as input and searches across four specialised knowledge bases: Tickers, Macro, Concepts, and Strategies.

These components and workflow functions are detailed in the sections below.

Finally, all outputs converge at the **explanation layer**. A prompt-building helper (`_build_explanation_prompt`) aggregates the risk metrics, RAG chunks, risk classification, and LSTM forecast into a single prompt for the LLM, which then generates the user-facing explanation. A post-generation hallucination guard (`_check_numbers`) checks that numerical claims remain consistent with the computed metrics before the response is returned.

4.2 User interface

The user interface is implemented in Streamlit and is designed to guide users from portfolio entry to chat-based analysis in a simple, structured flow.

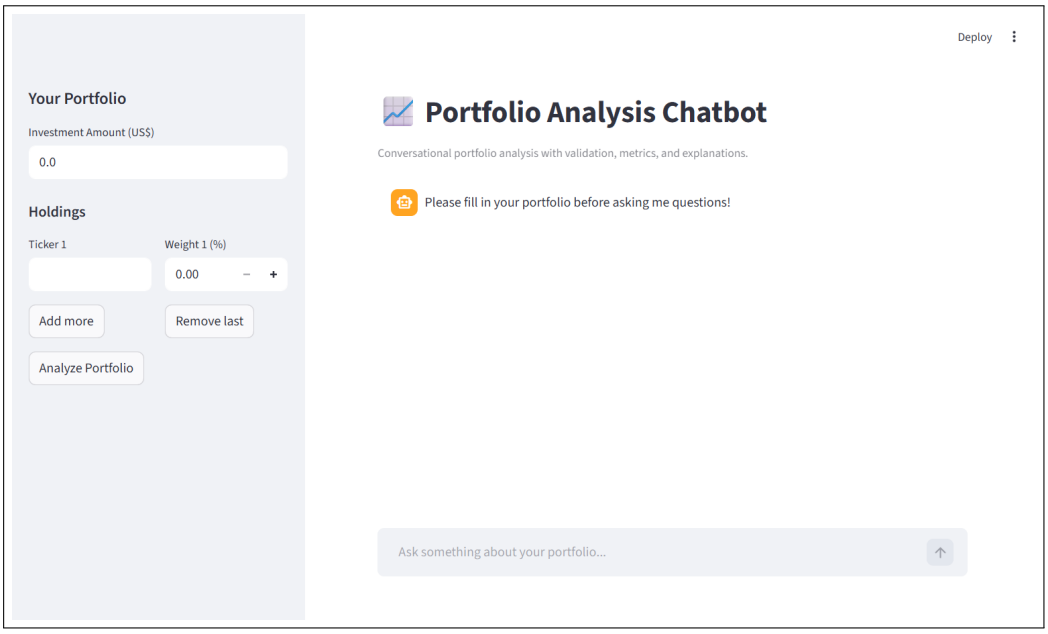

Figure 4: Initial interface of the portfolio analysis chatbot.

As shown in Figure 4, the sidebar is the primary interface for portfolio construction. Users enter tickers, weights, and total investment amount. Validation checks ensure that tickers are valid and that weights sum to 100%, while invalid inputs trigger explicit error messages. Tickers are normalised to uppercase, and the number of holding rows can be adjusted dynamically through add/remove controls.

The main interface is a chat-based analysis panel, shown in Figure 5, with clear visual separation between user and system messages. A loading indicator is displayed while backend processing is underway.

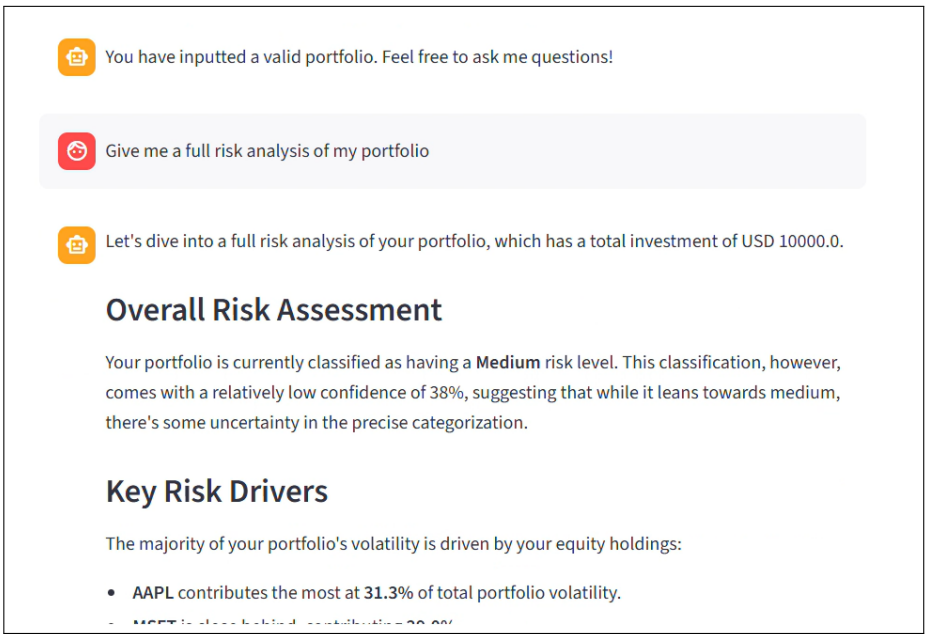

Figure 5: Chat-based query and response interface of the portfolio analysis chatbot.

A key design feature is chat locking. The chat input remains disabled until a valid portfolio has been submitted, preventing premature queries and enforcing a logical interaction flow.

Session state is managed through Streamlit's `st.session_state`, which preserves the current portfolio, validation status, chat history, cached outputs, and portfolio-update signals across reruns. This allows the interface to remain responsive without losing conversation context or analytical results.

Having outlined the user-facing interaction flow, the next subsection turns to the backend modules that validate inputs, execute analytics, and assemble the final response.

4.3 Key modules

This subsection explains the implementation module by module, following the same broad order used at runtime. It begins with the shared data layer, then the workflow and explanation layer, followed by the quantitative analytics module, the LSTM forecasting component, and the RAG module. This organisation helps the reader move from foundational preprocessing to higher-level reasoning and response generation.

4.3.1 Data Tools

The `agent_tools.data_tools` package implements the shared data layer used before any quant, LSTM, or orchestration step. Its role is to enforce a consistent input contract (valid symbols, valid weights, aligned adjusted prices, and return matrices), so downstream modules operate on the same canonical representation rather than re-implementing ad hoc preprocessing logic.

4.3.1.1 Module composition

The package exposes four core functions, each responsible for one part of the shared data pipeline.

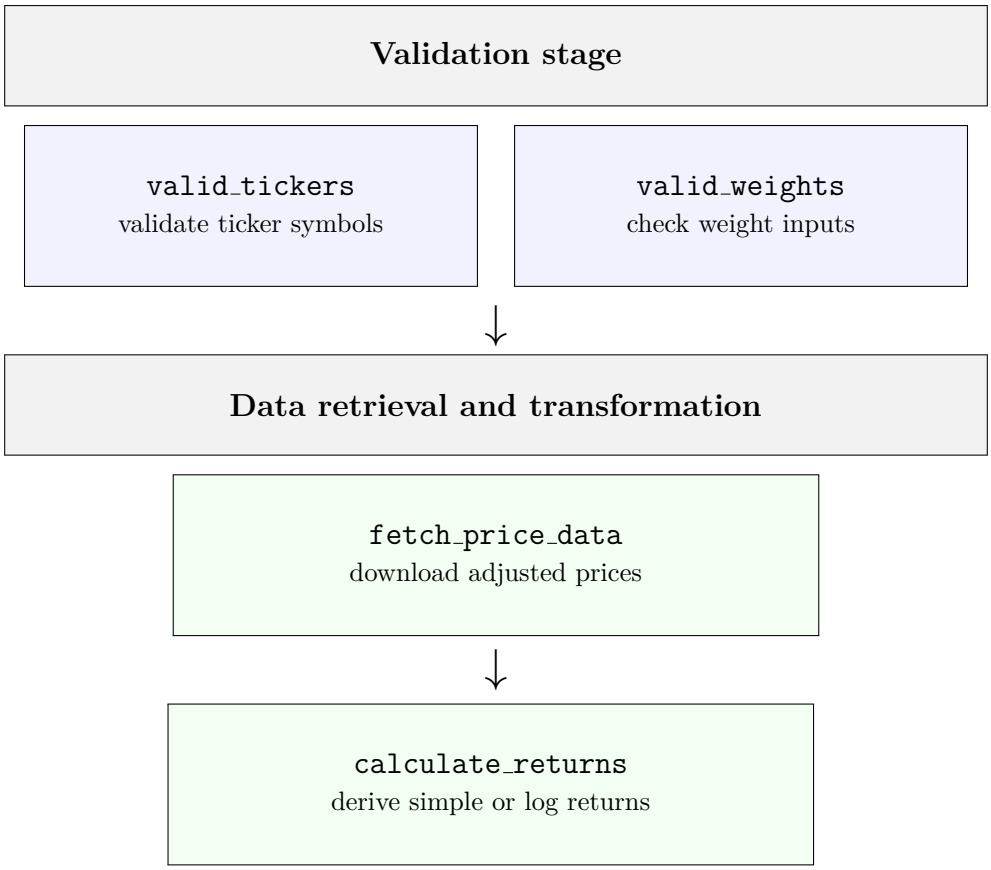

Figure 6: Core functions in the `data_tools` package.

4.3.1.2 `valid_tickers`

This function performs batched symbol checks against Yahoo Finance using a short recent window. Inputs are first normalised (trim, uppercase, remove empties), then fetched in one request for efficiency. A ticker is marked invalid if no recent close series is available. The output is a tuple `(ok, bad_symbols)`, allowing the UI and workflow layer to reject invalid portfolios early with explicit feedback.

4.3.1.3 `valid_weights`

This function enforces numerical sanity constraints on submitted weights:

- non-empty vector,

- all values finite,
- no negative entries,
- sum approximately equal to either 1.0 (fraction format) or 100.0 (percentage format), within tolerance.

It returns `(ok, message)` so validation errors can be surfaced in user-readable form.

4.3.1.4 `fetch_price_data`

This function downloads market data and standardises structure for the rest of the system. It explicitly extracts `Adj Close` (split/dividend-adjusted), handles both single- and multi-index `yfinance` outputs, sorts by date, removes timezone metadata, and returns a date-indexed matrix with ticker columns. Optionally, `SPY` can be auto-appended (`include_spy=True`) when absent, so benchmark-dependent analytics can run without an additional network call.

4.3.1.5 `calculate_returns`

This function converts aligned price levels into aligned return series using one of two controlled methods:

- simple returns: `pct_change()`,
- log returns: `log().diff()`.

Rows with missing values are dropped after transformation, preserving a common observation set across assets.

4.3.1.6 System-level rationale

Centralising these steps in one module gives three benefits:

1. **Consistency**: all downstream tools consume the same cleaned data conventions (ticker format, adjusted prices, return definitions).
2. **Reproducibility**: equivalent inputs produce equivalent matrices regardless of which workflow branch is executed.
3. **Testability**: each function has a narrow, typed I/O contract, so failures can be isolated at the data boundary before higher-level reasoning runs.

4.3.2 Main Workflow Tools

Every user message flows through two top-level functions in sequence:

```
classify_intent → route_and_execute
```

`classify_intent` is called first and independently. `route_and_execute` then runs all computation stages, assembles the `ExplanationContext`, calls `generate_explanation` internally, and returns a `WorkflowResult` to the caller.

Each function has a strict input/output contract, so each stage can be flexibly tested or replaced. The four computation tools (quantitative metrics, rule-based risk classifier, LSTM forecaster, and RAG) are treated as interchangeable black boxes from the pipeline's perspective.

4.3.2.1 `classify_intent`

Its role is to classify each user message into a primary intent, an optional secondary intent, and extracted entities before any computation runs. It uses `gemini-2.5-flash-lite` at temperature 0.0 because this step is executed on every message.

Pipeline

1. **Context assembly:** A structured prompt is built from three sources:
   - **portfolio_changed flag**, injected as a first-class signal so the model can distinguish between a textual query and a silent portfolio update.
   - **Recent conversation history**, with each message truncated to 200 characters. This helps the classifier resolve pronouns and follow-up references without uncontrolled prompt growth.
   - The **current message**, placed last so it remains closest to the model's attention focus.
2. **Schema-constrained LLM call:**
   `response_mime_type="application/json"` combined with `response_json_schema=IntentClassification.model_json_schema()` forces Gemini to **emit valid, typed JSON structures**, hence eliminating an entire class of parsing bugs rather than free-text JSON extraction with regex. The Pydantic schema is as such:

```
class IntentClassification(BaseModel):
    primary_intent:    str  # one of 6 intent names
    confidence:        float
    reasoning:         str
    secondary_intent:  Optional[str]
    extracted_metrics: Optional[list[str]]  # canonical metric keys
    extracted_concept: Optional[str]
```

3. **Fuzzy-match safety net:**
   The raw intent string is first resolved by direct enum lookup. If the model returns a variant such as "concept explaining", an **INTENT_ALIASES** map applies substring-based recovery. Unresolvable values fall back safely to `GENERAL_CHAT` instead of crashing the pipeline.
4. **Returns an `IntentResult` dataclass:**
   This object is consumed by `route_and_execute`. The `extracted_metrics` and `extracted_concept` fields propagate through to the explanation stage, where they control what information is surfaced.

6-Intent Taxonomy

The classifier uses six intents. This compact taxonomy improves routing stability and reduces overlap between labels. Rebalancing was initially treated as a separate intent, but it was merged into `FULL_ANALYSIS` because both follow the same computation path and the split caused confusion during testing.

Intents	Description
FULL_ANALYSIS	- **holistic risk assessment**, or - questions about changing **portfolio composition** - **comprehensive answer**, not a single number
SPECIFIC_METRIC	- **named or inferable metric** for the user's portfolio - requires `extracted_metrics` to be populated
CONCEPT_EXPLANATION	- educational, definitional, hypothetical - e.g. "what is X", "how does X work", "what if I had all tech stocks?"
TREND_PREDICTION	- forward-looking, triggers LSTM forecast tool
FOLLOW_UP	- clarification of prior response with no new topic - strictly distinguished from `CONCEPT_EXPLANATION`: "what do you mean by skewness?" vs `FOLLOW_UP`: "why did you say my skewness is excellent?"
GENERAL_CHAT	- greetings, thanks, capability questions, off-topic

Disambiguation Rules in the System Prompt

The classifier prompt includes explicit rules for ambiguous queries so that closely related intents are separated consistently. In particular:

- "What's MY Sharpe ratio?" → `specific_metric`.
- "What IS the Sharpe ratio?" → `concept_explanation`.
- Broad risk questions default to `full_analysis` unless a specific metric is named.
- Greetings remain incidental when a real query is present.

Entity Extraction

For `SPECIFIC_METRIC`, `extracted_metrics` must always be populated, even for vague phrasing. The system prompt therefore includes explicit inference mappings such as:

```
"downside" / "downside risk"   → [var_95, cvar_95, sortino_ratio]
"performance" / "returns"      → [sharpe_ratio, sortino_ratio]
"concentration"                → [hhi_concentration, avg_pairwise_correlation]
"tail risk" / "extreme events" → [excess_kurtosis, cvar_95]
"market sensitivity"           → [beta]
```

These keys match the output schema of `calculate_all_metrics`, so they pass directly into the orchestrator without an extra translation step. For `CONCEPT_EXPLANATION`, `extracted_concept` stores a plain-language concept name used to label and focus the explanation, while the full user query is still sent to concept-oriented RAG retrieval.

Key Design Decisions

- **Flash-Lite** for classification: heavier compute is reserved for explanation generation, where reasoning depth directly affects response quality.
- **Schema-constrained** output: one source of validation truth; no brittle regex. `temperature = 0.0`: deterministic classification matters when users are making financial decisions and expect consistent behaviour across repeated queries.
- **Fuzzy-match + `GENERAL_CHAT`** fallback: the pipeline never crashes on the classifier output drift. It degrades to a safe, helpful response.

- **Rate limit errors (`HTTP 429`)** are re-raised unconditionally so the `KeyRotator` in the caller can retry on the next API key. All other exceptions are caught and return the safe fallback.

4.3.2.2 `route_and_execute`

This function is the orchestration backbone. It receives the `IntentResult`, runs the required tools in the correct order, reuses cached results when possible, assembles an `ExplanationContext`, and calls `generate_explanation`.

Pipeline

1. **Intent-to-tool mapping** via set intersection. One line per flag with no branching:

```
_NEEDS_METRICS = {FULL_ANALYSIS, SPECIFIC_METRIC, TREND_PREDICTION}
_NEEDS_RISK    = {FULL_ANALYSIS}
_NEEDS_LSTM    = {FULL_ANALYSIS, TREND_PREDICTION}

intents       = {primary} | ({secondary} if secondary else set())
needs_metrics = bool(intents & _NEEDS_METRICS)
needs_risk    = bool(intents & _NEEDS_RISK)
needs_lstm    = bool(intents & _NEEDS_LSTM)
```

   The union of primary and secondary intents supports dual-intent messages in a single pass without special-case branching.

2. **Cache staleness check** (per stage):

```
metrics_stale = portfolio_changed OR
                (cache.get("metrics") is None)
risk_stale    = portfolio_changed OR
                (cache.get("risk_level") is None)
lstm_stale    = portfolio_changed OR
                (cache.get("trend_forecast") is None)
```

   A shallow copy of the prior-turn cache is made at the start of the function. Unchanged values are carried forward, while recomputed outputs overwrite only their corresponding keys. As a result, repeated follow-up questions on the same portfolio typically reuse price data, metrics, classifier outputs, and forecasts.

   `portfolio_changed` is the key invalidation signal. When the user submits a new portfolio, all stale stages recompute and comparison mode becomes available in the explanation layer.

3. Stages 1 - 4, **Tool execution** (conditional): Each stage runs only when its **needs_*** flag is True, as well as its ***_stale** flag is **True**. The four tools are listed in the table below. Their designs, workflows and performances are extensively detailed in the subsequent sections below.

Stages	Actions
1. Quantitative metrics	- price fetch + 12 scalar risk metrics, plus risk contribution - results stored in `cache["metrics"]` as `all_metrics` and `metric_analysis`
2. Rule-based risk classifier	- deterministic composite scoring - result stored in `cache["risk_level"]`
3. LSTM volatility forecast	- 60-day lookback forward pass - result stored in `cache["trend_forecast"]`
4. RAG retrieval	- runs only for intents that need external grounding; follow-up turns trigger it conditionally - when retrieval executes, the result is stored in `cache["rag_context"]`

4. **`ExplanationContext` assembly** (`_build_explanation_context`): This step converts the flat cache and intent output into the context dict used by the explanation layer. The main scope rules are:

   - `clean_metrics` omits the nested `risk_contribution` dict; that structure is passed separately.
   - `requested_metrics` is preserved so the response can stay focused on the metrics the user explicitly asked for.
   - If the portfolio changed and `old_cache` exists, prior-turn metrics, risk contributions, and risk classification are stored under `previous_*` keys.
   - Those `previous_*` fields enable **before/after comparison mode**, so the LLM can compare the old and new portfolios instead of producing a standalone reading.
   - Dual-intent queries populate the secondary fields, allowing both requests to be answered in one response.
   - `chat_history` keeps only the **last 6 messages**, with each message truncated to 300 characters.

5. API call with **key rotation**: The assembled context is passed through the key rotator before the explanation call:

```
        _key_rotator.call_with_retry(
            lambda client: generate_explanation(ctx, client=client))
```

   The `KeyRotator` manages up to 6 API keys. On a `429`, it **moves to the next key and retries automatically**. This keeps **rate-limit handling** out of both the orchestrator and the explanation logic.

Return Value

```
@dataclass
class WorkflowResult:
    content:          str
    intent:           Intent
    secondary_intent: Optional[Intent]
    cache:            dict   # full merged working_cache for this turn
```

The Streamlit session layer stores `WorkflowResult.cache` in `st.session_state` and passes it **back as the cache argument** on the next call to `route_and_execute`. The

cache is the mechanism that makes **multi-turn conversations about the same portfolio** efficient without any explicit session management logic inside the pipeline.

Key Design Decisions

- **Set-intersection for tool requirements**: clean, declarative, handles dual-intent messages without branching. Adding a new intent only requires updating the constant sets, not the execution logic.
- **Shallow-copy cache with per-key staleness**: stale data is never served; valid data is always reused. The copy ensures the prior turn's cache is never mutated in-place, keeping session state predictable.
- `portfolio_changed` as a **first-class pipeline signal**: it simultaneously invalidates the cache, triggers a full recompute, and activates comparison mode in `generate_explanation`. These behaviours are driven by a single boolean.
- Strict **separation between orchestration and generation**: the orchestrator controls what data the LLM sees and applies scope rules, while the LLM synthesises and narrates. Neither component is exposed to the other's internal logic.

4.3.2.3 `generate_explanation` (called inside `route_and_execute`)

Taking a deeper look at the final step of `route_and_execute`, `generate_explanation` converts the assembled `ExplanationContext` into the natural-language response using gemini-2.5-flash. This heavier model is justified because this step requires multi-document synthesis, financial domain reasoning, intent-specific tone and structure, all with strict numerical precision. Response quality is what the user directly experiences.

Pipeline

1. **Conditional prompt assembly** (`_build_explanation_prompt`): The context dict is serialised into a **multi-section markdown prompt**. Sections are included only when the corresponding data is present:
   - **Core sections**: the user's question and current portfolio.
   - **Analytical sections**: requested metrics, benchmarks, risk classification, risk contributions, LSTM output, prior-portfolio context, and RAG evidence.
   - **Dialogue sections**: concept name, secondary request, and recent conversation history.

   As a result, simple intents such as `GENERAL_CHAT` receive a compact prompt, while `FULL_ANALYSIS` in comparison mode receives a much richer prompt. The LLM is therefore exposed only to information relevant to the current intent.

2. `AUTHORISED NUMBERS` block (**hallucination grounding**): After all contextual sections, two grounding blocks are appended last in the prompt:

```
## AUTHORISED NUMBERS
<metric_key> = <value_to_4dp>
...

## GROUNDING REQUIREMENT
Use only numbers that appear verbatim above.
```

The authorised set is built from all scalar metrics, risk contributions, the composite risk score, LSTM forecast values, and portfolio weights, all stored to 4 d.p. Placing these blocks last in the prompt maximises their recency in the model's attention window and makes them the most immediate numerical constraint at generation time.

3. LLM call:

```
model:              gemini-2.5-flash
temperature:        0.0    - deterministic;
                                 no creative drift on numbers
max_output_tokens:  8192   - generous ceiling
                                 for full-analysis responses
ThinkingConfig(thinking_budget=1024)
```

A `thinking_budget` of `1024` tokens allows for a **private chain-of-thought** pass before the visible response is generated. This is most valuable for:

- **comparison mode**, where the model must compute before/after deltas across two full scalar-metric sets, and
- **dual-intent** messages, where it must structure two coherent answers from a shared data pool without repeating information.

This private reasoning does not appear in the user-facing output, so it improves coherence without increasing visible response length.

4. LLM call: Post-generation **hallucination audit** (`_check_numbers`): After the response is received, a regex **extracts all decimal numbers** and checks each **against a pre-built authorised set**. To avoid false positives, each authorised value is stored in six representations:

```
f"{v:.4f}"        0.2234  - canonical
f"{v:.2f}"        0.22
f"{v:.1f}"        0.2
f"{v*100:.2f}"    22.34   - percentage form
f"{v*100:.1f}"    22.3
f"{v*100:.0f}"    22
```

Suspect numbers are **logged as warnings**; the response is returned regardless. This is intentionally a development **debugging aid**, not a production gate. Numbers like "over the last 5 years" or "a beta of 1.2 means..." appear in narrative context and are not fabrications. The primary grounding mechanism is the prompt-level `AUTHORISED NUMBERS` block; **_check_numbers** surfaces potential issues during development without blocking users on false positives.

Intent-specific Output Contracts (`explain_system_prompt`)

The system prompt defines a **distinct output structure per intent**. This prevents the LLM from defaulting to a generic "here is everything" format regardless of what was asked:

Key Design Decisions

- **Separate model from the classifier**: response quality is a user-facing output, heavier model is used here.

Intent	Main structure	Special rule or emphasis
`FULL_ANALYSIS`	Standalone: overall risk level; key risk drivers; notable metrics; trend outlook if present; considerations framed as observations, not advice.	If `portfolio_changed=True`, switch to comparison mode: what improved, what worsened, how the risk classification shifted, which assets now drive risk, and whether the change appears beneficial.
`SPECIFIC_METRIC`	Current value and what it means; benchmark label and comment; brief relevance to this portfolio.	Stay narrowly focused on the requested metric rather than expanding into a full analysis.
`CONCEPT_EXPLANATION`	Plain-language definition; why it matters for portfolio risk; simplified explanation of how it is calculated.	Use the user's real portfolio numbers as an illustration when they are available.
`TREND_PREDICTION`	What the forecast predicts; comparison of predicted volatility with current volatility; what could invalidate the prediction; explicit uncertainty disclaimers.	Do not conflate `predicted_direction` with volatility direction. "Up" refers to positive predicted returns, not rising volatility.
`FOLLOW_UP`	Concise answer to exactly what was asked.	Reference the prior response without re-summarising the full earlier analysis.
`GENERAL_CHAT`	Friendly response with a short capabilities summary.	Politely redirect off-topic questions back to portfolio analysis.
Dual intent	Lead with the primary intent's answer, then provide the secondary answer.	Use a clear section break such as `---`, and avoid repeating data already given in the primary answer.

- **`AUTHORISED NUMBERS` + `GROUNDING REQUIREMENT` placed last**: recency in the context window maximises their influence as a constraint at generation time.
- **Multi-representation authorised set**: handles percentage vs. decimal presentation without flagging legitimate formatting choices as hallucinations.
- **ThinkingConfig(budget=1024)**: private reasoning improves structural coherence on complex tasks (comparison mode, dual-intent synthesis) without increasing the length of the response the user reads.
- **Conditional prompt sections**: the prompt is always as short as the task requires and no shorter. Simple intents do not pay the token cost of complex ones.

4.3.3 Quant Tools

The quant module is the computation layer of the pipeline. It receives pre-fetched return data from the data layer and produces the risk metrics consumed by both the classifier and the explanation layer. It does not fetch or clean data, and remains stateless and deterministic.

4.3.3.1 Inputs

The orchestrator calls the module's primary wrapper as such:

```
all_metrics = calculate_all_metrics(
    returns=returns,
    weights=portfolio["weights"],
)
```

The returns DataFrame is produced by the data layer's `fetch_price_data` function, containing daily simple returns for all portfolio assets with SPY appended as the market benchmark for beta calculation. The weights list corresponds to the user's assets only, excluding SPY.

4.3.3.2 Covariance Matrix

The covariance matrix is annualised from daily simple returns and computed once at the start of the pipeline. It is then reused for portfolio volatility, average pairwise correlation, and risk contribution, ensuring consistency while avoiding redundant computation.

4.3.3.3 Risk Metrics

The module computes 12 scalar metrics grouped into six clusters: volatility, loss threshold, drawdown, risk-adjusted return, return shape, and portfolio structure.

The volatility cluster covers both total risk, via `portfolio_volatility`, and its stability over time, via `vol_of_vol`. The loss-threshold cluster captures tail risk through `var_95` and `cvar_95`. The drawdown cluster uses `max_drawdown` computed from compounded wealth. Risk-adjusted return is measured through Sharpe and Sortino ratios, while return-shape diagnostics are captured by skewness and `excess_kurtosis`. The portfolio-structure cluster includes beta, concentration via `hhi_concentration`, and average co-movement via `avg_pairwise_correlation`. Per-asset `risk_contribution` is produced alongside these scalar metrics as a separate breakdown.

4.3.3.4 Benchmark Labelling

To make the metrics usable in the LLM prompt, each scalar metric is mapped by `metric_benchmarks` to a label and a plain-English comment derived from heuristic thresholds. For example, a Sharpe ratio below 0.8 is labelled "poor". These benchmark annotations give the explanation model interpretable context instead of leaving it to infer meaning from raw numbers alone.

4.3.3.5 Outputs

The module produces two outputs: six selected scalar metrics for the risk classifier and the full explanation bundle of 12 scalar metrics plus per-asset `risk_contribution`. The classifier uses `portfolio_volatility`, `max_drawdown`, `var_95`, `hhi_concentration`, `avg_pairwise_correlation`, and `sharpe_ratio` to compute a weighted composite score mapped to low, medium, or high risk. The explanation layer receives all scalar metrics together with their benchmark labels, while `risk_contribution` is passed separately as a per-asset breakdown.

4.3.3.6 Caching

The orchestrator uses a trailing five-year data window. After the first computation, metrics and benchmark analysis are cached for the session and reused whenever the portfolio is unchanged.

4.3.4 LSTM Tools

The main quant pipeline is backward-looking, so the system complements it with an LSTM-based Future Portfolio Risk Tool that adds a forward-looking signal. In the deployed chatbot, this module consumes the most recent 60-day log-return window for the submitted portfolio and returns four fields: a forecast volatility level, a directional signal (`Up`/`Down`), the probability of an upward move, and a confidence score.

These outputs are used only as an auxiliary forecast layer inside the orchestration pipeline. In particular, `predicted_direction` refers to the expected direction of **portfolio returns**, whereas `predicted_volatility` refers to the expected future volatility level. Keeping those two outputs separate prevents the explanation layer from conflating return direction with risk direction.

To train the model, a synthetic portfolio dataset is constructed from assets spanning equities, fixed income, commodities, energy, and real estate. This broad asset mix exposes the model to portfolios ranging from conservative to aggressive, rather than tying it to one narrow market segment.

Synthetic portfolios are generated with concentrated but realistic weight patterns, using a Dirichlet distribution so that a few assets receive larger weights while the remainder receive smaller allocations. From these portfolios, 60-day log-return sequences are extracted and paired with forward-looking volatility and return-direction labels defined by the training pipeline.

The training data pipeline uses non-overlapping windows to reduce leakage, removes ambiguous direction labels, and compensates for class imbalance with weighted binary cross-entropy so that downward moves are not ignored.

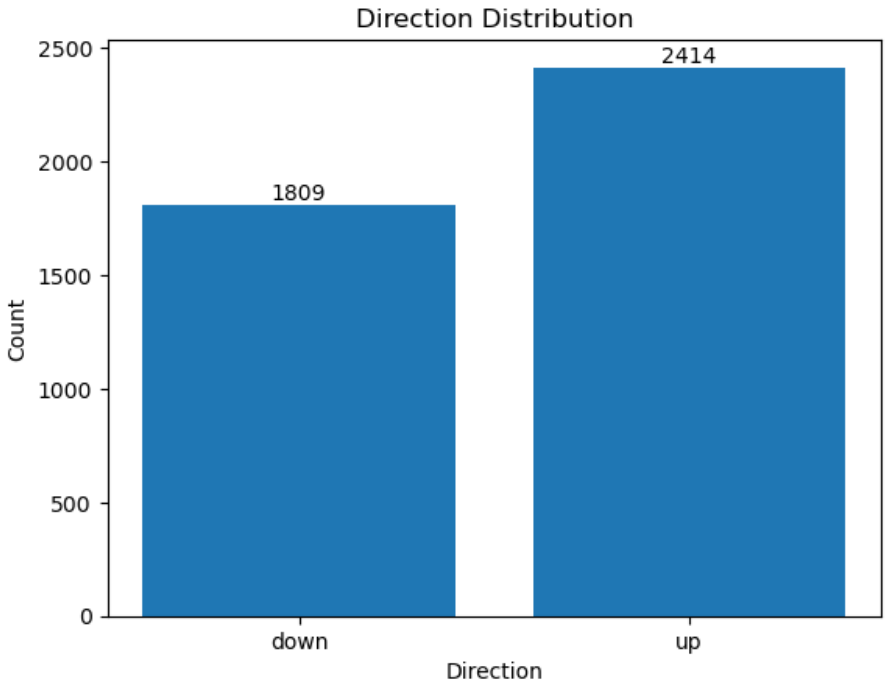

Figure 7: Bar plot showing class imbalance in synthetic dataset.

A two-layer LSTM architecture is used, with dropout for regularisation and separate output heads for volatility regression and directional classification. The model is trained with mean squared error for the volatility head, binary cross-entropy with logits for direction, and Adam optimisation with weight decay for stability and generalisation.

After training, the learned weights are reused for inference inside the chatbot. A lightweight preprocessing pipeline converts the user's portfolio into the log-return input required by the model, and the resulting forecast is passed to the orchestrator as an additional context signal rather than a standalone decision rule.

The main limitation is that synthetic training data cannot capture every real market regime, and the forecast does not directly encode macro shocks or breaking news. For that reason, the system complements the LSTM with RAG-based contextual retrieval so that forward-looking numerical signals can be interpreted alongside external context.

4.3.5 RAG Tools

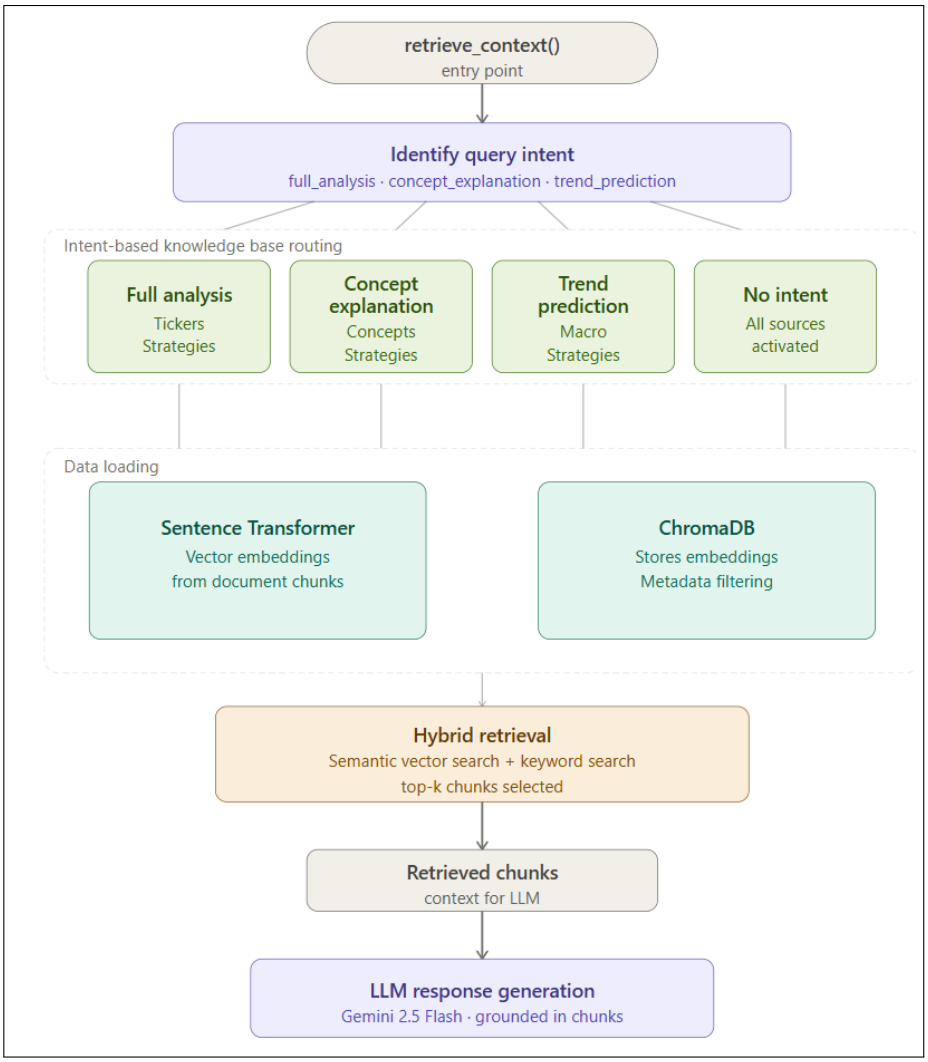

Figure 8: Overview of the RAG Retrieval pipeline.

Retrieval-Augmented Generation (RAG) provides the system's external knowledge layer, allowing the explanation model to reason over retrieved evidence instead of relying only on parametric memory. In this chatbot, RAG is not a standalone component; it is called selectively by the workflow when an intent requires external grounding.

As shown in Figure 8, retrieval begins with intent-based source selection. The mapping is:

- `full_analysis`: ticker and strategy stores;
- `concept_explanation`: concept and strategy stores;
- `trend_prediction`: ticker and macro stores;
- fallback: all stores.

This routing reduces noise by limiting retrieval to the sources most relevant to the user's question.

Before retrieval, the pipeline refreshes dynamic sources when they become stale. Ticker profiles are regenerated only for the resolved symbols needed by the current query, while the macro store refreshes FRED- and market-based data on a staleness schedule. All source documents are then converted into chunked text with citation metadata and grouped into persistent ChromaDB collections.

For ranking, the pipeline uses hybrid retrieval rather than pure vector search. BM25 is applied across the full chunk pool to capture exact lexical matches, while dense retrieval uses SentenceTransformer embeddings and ChromaDB similarity search to capture semantic relevance. The final ranking score is a weighted combination of the normalised BM25 and vector scores, which is especially useful in financial queries where exact ticker strings and broader semantic context both matter.

Retrieved candidates then pass through post-processing. Chunks below a minimum relevance threshold are filtered out, and surviving chunks are guaranteed to carry citation identifiers and source metadata before they are returned to the explanation layer.

The pipeline also computes Recall@k and Mean Reciprocal Rank (MRR) as runtime diagnostics, surfacing flags when relevant chunks are ranked poorly. These signals are primarily used to monitor retrieval quality and tune the pipeline, rather than to block user-facing responses.

After filtering and ranking, the retrieved chunks are inserted into the explanation prompt alongside the user query and computed portfolio context. The LLM therefore acts as a reasoning layer over fresh, query-specific evidence, which helps the system stay grounded without retraining the language model itself.

Taken together, the system is designed as a deterministic computation pipeline with modular analytical components and a constrained explanation layer. Numerical results are produced by fixed tools, reused through caching, and only then interpreted by the LLM, which improves reliability while preserving conversational flexibility.

5 Evaluation

5.1 LSTM Evaluation

Volatility Prediction Evaluation MSE	0.0026
Volatility Prediction Evaluation MAE	0.0389
Direction Prediction Evaluation AUC	0.5358
Direction Prediction Evaluation Accuracy	0.67

Table 2: Prediction results of volatility and future direction of portfolio on test dataset

As seen from Table 2, the model has relatively strong accuracy in predicting future volatility of portfolio over the next 10 days, with a test MSE of 0.0026 and MAE of 0.0389. These results suggest that the model is able to effectively learn and estimate short-term portfolio risk. This result is consistent with expectations, as volatility exhibits temporal clustering, making it more predictable from historical data. Comparatively, the model struggled in predicting future direction. The low AUC of only about 0.5358 suggests that the model's ability to distinguish between future upward and downward movements of the portfolio is only marginally better than random guessing.

	Future Upward direction	Future Downward direction
Precision	0.81	0.64
Recall	0.34	0.94
F1-score	0.48	0.76

Table 3: Classification report of future direction of portfolio on test dataset

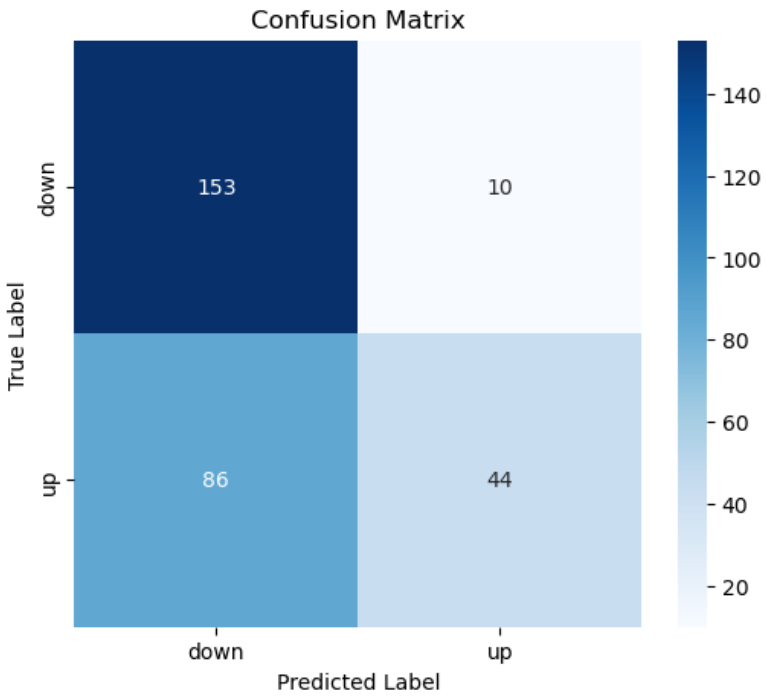

Figure 9: Confusion matrix on future direction of portfolio on test dataset.

As observed from the confusion matrix in Figure 9, the model performs better in predicting future portfolio downward movements. From Table 3, the model is observed to have very high recall for downward movements, showing that it is able to identify most future downward portfolio directions. However, it has low recall for future upward movements and many future upward trends are misclassified as downward. The model struggles to learn a robust directional signal and the input feature of 60 days log-return does not contain sufficient predictive information for future direction.

The discrepancy between volatility and direction prediction performance highlights the challenges in financial time series modeling. Volatility is generally easier to predict due to its relatively stable statistical properties like clustering, which allows the model to effectively learn temporal dependencies. Comparatively, future portfolio direction is inherently noisy and less predictable, especially when using relatively short input windows such as 60 days of log returns.

The model is also observed to be more conservative when predicting upward movements as seen from the significantly lower recall for upwards prediction. There are fewer false positives as seen from the higher precision for upwards prediction and the model is more sensitive in predicting downward movements with fewer missed negatives from the higher recall, which is relatively more desirable than the converse in a risk management context.

Therefore, while the LSTM is effective for volatility forecasting, it is less suitable for

directional prediction as it struggles to extract meaningful signals for direction due to the high level of noise and weak predictability in short-term portfolio direction.

5.2 RAG Retrieval Evaluation

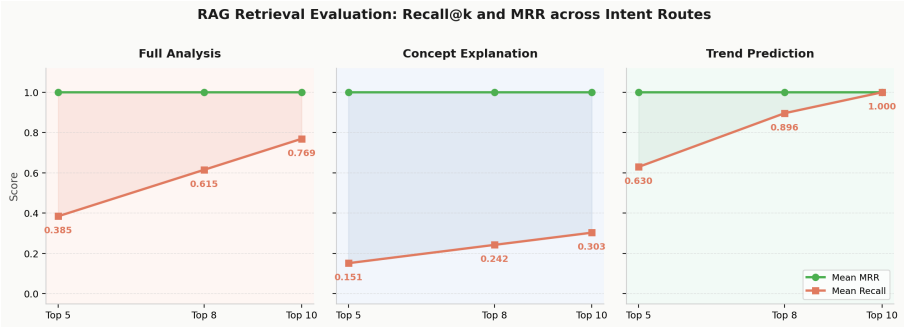

Figure 10: Recall and MRR at $k \in \{5, 8, 10\}$ across three intent routes.

To evaluate retrieval quality, nine queries were designed spanning all four intent routes - `full_analysis`, `concept_explanation`, `trend_prediction` and `no_intent`, at three difficulty levels: Easy, Medium and Hard. Retrieval performance was assessed using Recall@k and Mean Reciprocal Rank (MRR) at $k \in \{5, 8, 10\}$. While MRR measures the rank position of the first relevant result, Recall@k measures how many relevant chunks are surfaced within the top k; the two metrics are therefore complementary, a system can rank its first retrieved chunk perfectly (MRR = 1.0) while still missing out other relevant material (Recall $<$ 1.0).

As shown in Figure 10, MRR equals 1.0 across every intent at every value of k, indicating that whenever a relevant chunk is retrieved, it is consistently ranked first. This confirms that the hybrid retrieval strategy produces well-ordered results.

For `full_analysis` queries, Recall@k rises from 0.385 at k=5 to 0.769 at k = 10, reflecting consistent retrieval from a well-populated knowledge store. `Trend_prediction` achieves the strongest recall trajectory, reaching near-perfect scores at k=10, suggesting that dense semantic search handles indirectly phrased forward-looking queries effectively.

`Concept_explanation` records the lowest Recall@k values; however, this is an artefact of knowledge base construction rather than retrieval failure. Only one relevant chunk exists per concept in the concept store, placing a hard ceiling on the system and motivating future work to expand the concept corpus.

Across all intents, increasing k consistently improves Recall, confirming that the relevant chunks exist in the index and that broader retrieval windows surface them reliably. Overall, the results suggest that the pipeline is well-calibrated for ranking quality, with the primary path to improvement lying in expanding the concept knowledge store rather than in changes to the retrieval mechanism itself.

5.3 LLM Evaluation

Finally, we evaluated the workflow functions to verify consistent LLM performance, as these models are prone to variable outputs.

5.3.1 `intent_classifier`

5.3.1.1 Evaluation Methodology

The classifier was evaluated using a hand-crafted suite of 57 test cases organised into nine categories of increasing difficulty. These categories were designed to test not only straightforward intent recognition, but also disambiguation, dual-intent handling, contextual inference, and entity extraction under noisy or ambiguous phrasing.

- **Section 1 - Clear-cut intents:** unambiguous examples of all six intents, used to establish baseline performance on straightforward inputs.
- **Section 2 - Disambiguation:** lexically similar phrasings that should map to different intents, such as possessive metric queries versus definitional concept queries.
- **Section 3 - Multi-intent messages:** messages containing two distinct requests, requiring both primary and secondary intents to be set correctly.
- **Section 4 - Disguised intents:** cases where greetings or filler phrases wrap an actionable request and should not be misrouted as `general_chat`.
- **Section 5 - Ambiguous or stress cases:** vague, minimal, or highly underspecified messages that require the classifier to rely on scope and context.
- **Section 6 - Entity extraction:** cases that test whether metric and concept names are extracted correctly in addition to the intent label.
- **Section 7 - Hard multi-intent edge cases:** messages where two apparent requests collapse into one intent, or where secondary intents involve difficult cross-intent pairings.
- **Section 8 - Adversarial or noisy phrasing:** inputs containing negation, informal spelling, long messages with buried requests, or hypothetical phrasing that resembles another intent class.
- **Section 9 - Boundary between `full_analysis` and `specific_metric`:** cases targeting the hardest decision boundary in the taxonomy, where the distinction lies primarily in scope rather than wording.

Each test case was checked for:

1. correct primary intent,
2. correct secondary intent where expected, and
3. correct entity extraction where specified.

5.3.1.2 Results

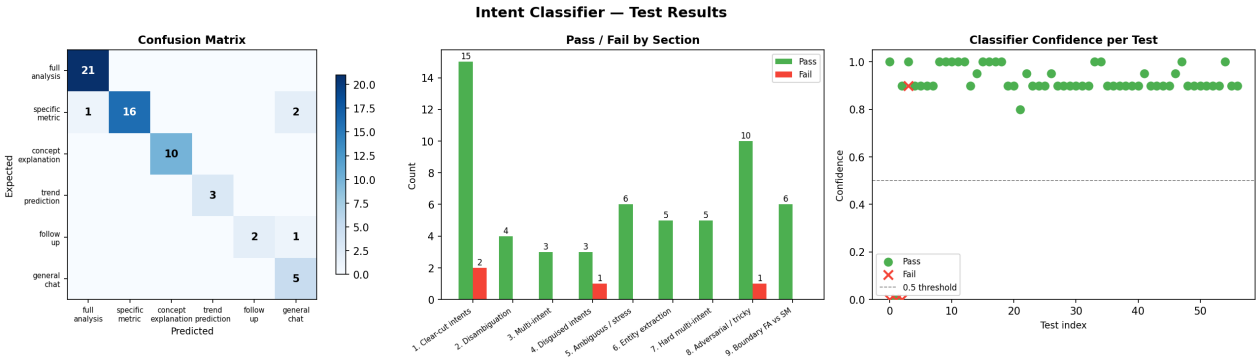

Figure 11: Plotted results of `intent_classifier` tests.

Overall, the classifier achieved approximately 85-88% accuracy (around 49-50 of the 57 test cases passed). By intent category, `full_analysis` achieved the highest absolute count with 21 correctly classified instances, making it the largest intent class, followed by `specific_metric` with 16 and `concept_explanation` with 10. The remaining categories recorded smaller counts. The confidence scatter plot showed a useful separation: correctly classified tests typically had model confidence of 0.8 to 1.0

It should also be noted that most failures clustered at 0, suggesting that misclassification might also be due to API failures (likely `503` error). Because when such errors strike, `intent_classifier` is designed to default the query to `general_chat`. The three test cases misclassified as `general_chat` in the rightmost plot, and their near-consecutive occurrences, support this inference.

5.3.2 `generate_explanation`

5.3.2.1 Evaluation Methodology

The explainer was evaluated using an LLM-as-judge framework across a single, balanced portfolio configuration (AAPL, MSFT, SPY, TLT, GLD, VNQ), covering all six intent types. The evaluation had two layers:

- Layer 1 — Full pipeline execution:
  For each intent type, a realistic test query was sent through the complete pipeline: `classify_intent` → `route_and_execute` → `generate_explanation`
  This tests the end-to-end behaviour, ensuring the explainer works compatibly in the full pipeline. The `follow_up` test case was chained off the actual `full_analysis` response to simulate a real conversational sequence.

- Layer 2 — LLM-as-judge scoring:
  Two independent models from different providers, Claude Sonnet (claude-sonnet-4-6) and Groq Llama (llama-3.3-70b-versatile), were chosen to guard against self-grading bias.

  They scored the responses on four dimensions, on a 0-5 scale:

1. **Factual accuracy**: Do the numbers and labels in the response match the provided data exactly?
2. **Intent alignment**: Does the response directly and fully answer the user's query?
3. **Completeness**: Does the response cover all required aspects for this intent type?
4. **Clarity**: Is the response well-structured, written in plain, comprehensible language?

5.3.2.2 Results

Results are visualised as dual heatmaps: rows represent intent types, columns represent the four evaluation dimensions, with one heatmap for each judge.

The initial evaluation (Figure 12) revealed concentrated factual-accuracy weaknesses in the `trend_prediction` and `full_analysis` responses. These failures were most plausibly driven by two issues: first, confusion between `predicted_direction` (direction of portfolio returns) and `predicted_volatility`; second, unsupported numerical claims in intents that referenced many metrics simultaneously.

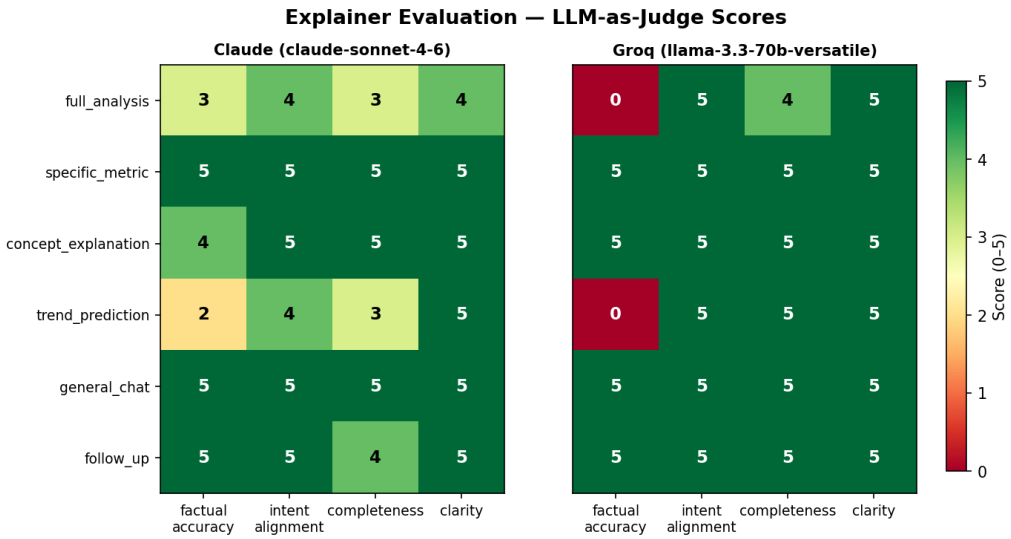

Figure 12: Initial judge scorings for the various responses.

To address these weaknesses, three targeted prompt changes were introduced:

- an explicit definition stating that `predicted_direction` refers to the direction of `portfolio returns`, not volatility;
- an inline `AUTHORISED NUMBERS` block in the `generate_explanation` prompt to constrain numerical references to provided values; and
- reducing Gemini 2.5 Flash temperature from `0.7` to `0.0` to improve determinism.

The final evaluation (Figure 13) shows clear improvement after these interventions, particularly in factual accuracy for the previously weak intents. This suggests that the

main issue was not the overall workflow design, but insufficient prompt-level grounding and field disambiguation in numerically dense responses.

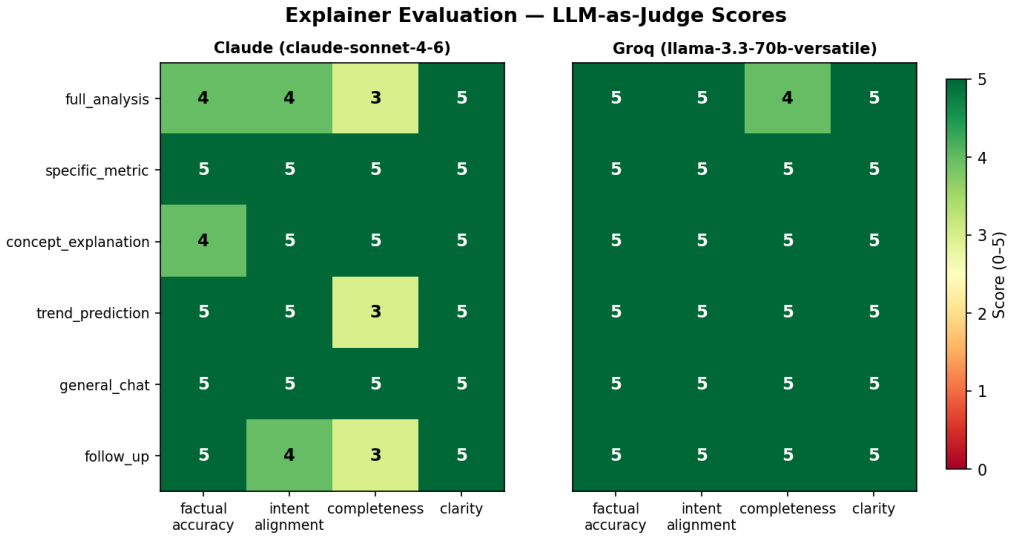

Figure 13: Final judge scorings for the various responses.

6 Discussion

This section reflects on the system from four complementary perspectives: implementation strengths, practical weaknesses, broader limitations, and responsible use. The discussion focuses on how these considerations arise from the actual design choices described in the preceding sections.

6.1 Strengths

6.1.1 Data Preprocessing

The preprocessing pipeline is a strength because it enforces a consistent data contract before downstream analytics run. Adjusted closes reduce split and dividend artefacts, bounded forward fill plus row dropping preserve an aligned trading calendar, and automatic `SPY` insertion and ticker normalisation improve robustness for benchmark-dependent metrics.

6.1.2 Architecture

The key architectural strength is the explicit separation between intent resolution and deterministic execution. Intent is inferred by the classifier, but execution order is enforced in `route_and_execute()` through fixed intent-to-tool membership sets, rather than unconstrained LLM planning. This guarantees dependency order for metric computation (for example, covariance before derived portfolio statistics), improving reliability and debuggability. A session-level cache keyed by a portfolio hash avoids redundant recomputation when holdings are unchanged. The numeric hallucination guard further provides defence in depth: authorised numeric values are injected into the prompt and post-generation regex checks flag numbers not matching computed results. The design is also cost-aware:

`gemini-2.5-flash-lite` is used for frequent structured classification, while a more capable model is reserved for explanation generation.

6.1.3 Quantitative Risk Module

The quantitative module computes 12 risk metrics spanning multiple dimensions of risk exposure (volatility, tail loss proxy, drawdown, risk-adjusted return, concentration, correlation, and related diagnostics). A covariance matrix is computed once and reused across dependent metrics, reducing duplication and ensuring consistency. Metrics are paired with benchmark labels so explanations remain interpretable instead of presenting isolated raw values.

6.1.4 LSTM Forecasting Module

The LSTM component shows strong test performance for volatility prediction (MSE ≈ 0.0026). Its directional behaviour is asymmetric in a risk-appropriate way: higher precision on upward signals and higher recall on downward signals, which is desirable when missed downside warnings are costlier than false alarms. Weighted cross-entropy during training also reduces bias toward majority directional classes.

6.1.5 RAG Retrieval Module

The RAG pipeline achieves strong retrieval ordering performance (MRR = 1.0 across evaluated intents), indicating that relevant chunks are consistently ranked at the top when available. Hybrid retrieval (dense + lexical with reciprocal rank fusion) and intent-conditioned routing improve precision by restricting search to relevant stores. For trend-style intents, high Recall@k at larger k suggests the semantic channel can still surface useful context for less direct forward-looking queries.

6.1.6 User Interface

The interface enforces a guided flow through ticker/weight validation, automatic ticker normalisation, and chat-locking before a valid portfolio is submitted. These checks reduce malformed requests reaching backend tools. Immediate feedback messages and loading cues increase transparency during processing and improve usability.

6.2 Weaknesses

6.2.1 Data Preprocessing

A practical weakness is reliance on vendor-managed historical data: Yahoo Finance may revise adjusted price histories over time, so regenerated datasets can differ across runs even when the code is unchanged. In addition, the quantitative pipeline uses simple returns whereas the LSTM pipeline uses log returns. Although this choice is defensible, it reduces cross-module transparency and makes direct comparison less intuitive.

6.2.2 RAG Retrieval Model

The RAG pipeline relies on a relevance threshold. When no chunk passes threshold, fallback top-k context may still be passed to generation, increasing risk of weak grounding.

Embedding freshness is another fragility: if stores are updated while refresh is deferred, retrieval can lag current content. Operationally, vector-store persistence errors can also interrupt retrieval without rich user-facing recovery. Finally, uneven store population (for example, sparse concept chunks) can cap attainable Recall@k irrespective of retriever quality.

6.2.3 User Interface

Reasoning visibility remains mostly backend-facing; users do not see intermediate tool calls or retrieval traces unless additional surfacing is implemented. Session state is memory-backed and reset on refresh, so chat and portfolio continuity are not persisted across browser sessions. The strict chat-locking policy is safe but rigid for exploratory users who may want non-portfolio explanatory queries before full submission.

6.2.4 Explanation Layer

The hallucination guard currently flags suspicious numbers but does not automatically suppress or regenerate responses, so remediation is incomplete. Conversation context is truncated to a short recent window, which may degrade coherence for long sessions. Ambiguous multi-intent queries can also lose secondary intent when classifier confidence is low, resulting in incomplete context assembly for generation.

6.3 Limitations

6.3.1 Data Preprocessing

The preprocessing layer is also limited by its data source and scope. Dependence on `yfinance`/Yahoo Finance creates a single external point of failure, while the use of end-of-day adjusted closes excludes intraday dynamics, bid-ask effects, and liquidity stress. The default window beginning at `2020-01-01` captures recent regimes but not longer historical cycles, and survivorship effects are not explicitly modelled.

6.3.2 Architecture

The system is scoped to long-only portfolios and excludes options, leverage, and short positions. Instrument coverage is constrained by vendor availability. Historical-metric interpretation implicitly assumes a stable holding period over the sampled window, which may not match recently constructed portfolios. Conversation and comparison memory are also bounded by design.

6.3.3 LSTM Forecasting Module

The LSTM is stronger on volatility magnitude than directional forecasting under noisy short-horizon returns. Training data are largely synthetic portfolio constructions, which may not fully capture real deployment distributions. Macro signals (rates, regime indicators, event features) are not explicit model inputs, limiting robustness to abrupt regime shifts.

6.3.4 RAG Retrieval Model

The embedding model is general-purpose rather than finance-tuned, so domain-specific terminology may be underrepresented in vector space. A cross-encoder reranker is not currently in the retrieval stack.

6.3.5 Explanation Layer

Both Gemini models used are general-purpose, not financial-domain-specialised. Multi-intent handling exists but depends on secondary-intent emission quality, which can fail silently under ambiguous phrasing.

6.4 Ethics and responsible use

This system is a research prototype and does not provide regulated financial advice. Outputs should therefore be interpreted alongside professional judgement rather than treated as standalone recommendations. While retrieved citations and numeric grounding checks improve transparency, they do not eliminate model risk or guarantee factual correctness under all conditions. Reported values are historical-statistical descriptors under explicit assumptions, not forward-looking guarantees.

7 Conclusion

7.1 Key Outcomes

This project shows that a hybrid architecture, combining deterministic quantitative workflows with LLM-based language generation, can produce a coherent and defensively designed portfolio risk assistant within a prototyping scope. The system computes a broad set of risk metrics from live market data, grounds narrative responses in computed values with a post-generation hallucination check, and retrieves contextual financial knowledge through hybrid lexical-semantic search. The LSTM module contributes forward-looking volatility signals, while intent-conditioned RAG routing improves retrieval precision across query types. Overall, the implementation demonstrates how reliability-critical financial tooling can retain natural language flexibility without surrendering execution control to unconstrained generation.

7.2 Possible Future Improvements

7.2.1 Architecture

Future work could extend support beyond Yahoo Finance to include additional data sources, broadening the chatbot's applicability to niche equities and cryptocurrency instruments currently excluded from the pipeline. Support for more complex portfolio structures, such as short positions, leveraged products, and options, would also allow the system to serve a wider range of investment strategies.

Replacing session-based storage with a persistent database layer would allow portfolio history, chat records, and analytical outputs to be retained across sessions, enabling longer-term tracking and multi-session continuity.

7.2.2 LSTM Forecasting Module

The forecasting module could be improved by incorporating macroeconomic signals such as interest rate data, volatility indices, and yield curve information, since log returns alone are too noisy for robust directional learning. Additional input features would improve the model's ability to anticipate regime shifts and market shocks that are not detectable from the past 60 days of return history alone. Training on real-world portfolio data, rather than synthetically generated sequences, would also improve generalisation across a broader range of portfolio compositions.

7.2.3 RAG Retrieval Model

As mentioned in our limitations, expanding the concept knowledge base beyond its current single-chunk-per-concept structure would increase the hard ceiling on Recall@k for concept explanation queries. Adding a cross-encoder re-ranker as a post-retrieval stage would improve chunk ordering before generation, tackling the issue of the current reliance on embedding similarity alone. Fine-tuning the embedding model on financial language would improve retrieval precision for domain-specific terms that general-purpose sentence transformers do not represent well.

7.2.4 Explanation Layer

To improve handling of ambiguous multi-intent queries, the current prompt-based intent classifier could be replaced with a fine-tuned BERT model, since the zero-shot LLM classifier occasionally settles on a single route and omits relevant context. Upgrading to a more capable language model better suited to financial reasoning would also be appropriate for any client-facing deployment beyond the testing stage.

7.2.5 User Interface

To improve transparency, the interface could surface the system's reasoning steps during loading, showing which tools were called and what context was retrieved before the LLM generated its response. An expandable section in the final output could allow users who want deeper insight to inspect this reasoning trace without cluttering the default view.

7.3 Closing Remarks

In summary, this work presents a portfolio risk analyst chatbot that combines a structured quantitative backend, a retrieval-augmented knowledge layer, and an LLM explanation interface. A central design principle is the separation of intent resolution from deterministic execution, which improves reproducibility, observability, and debugging compared with fully autonomous tool planning. The post-generation numeric guard further addresses a critical failure mode in financial LLM responses by checking generated values against computed metrics. The most meaningful next steps are replacing synthetic LSTM training distributions with real portfolio histories, improving retrieval quality with finance-adapted representations and reranking, and adding persistence for cross-session continuity. These extensions would move the system from a strong research prototype toward production-ready decision support.